

Machine Learning Pipeline

From Data to Deployment

Kiran Bagale

IOE, Thapathali Campus

December, 2025

Outline

- 1 Introduction to ML Pipeline
- 2 Preprocessing and Cleaning
- 3 Model Development
- 4 Training, Testing, and Hyperparameter Tuning
- 5 Summary

What is a Machine Learning Pipeline?

Definition

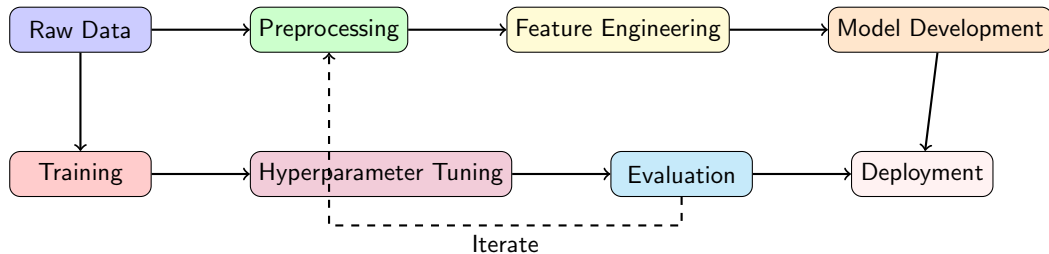
A Machine Learning Pipeline is an end-to-end workflow that automates the process of building, training, and deploying machine learning models.

Key Components:

- 1 Data Collection and Integration
- 2 Data Preprocessing and Cleaning
- 3 Feature Engineering
- 4 Model Development
- 5 Training and Validation
- 6 Hyperparameter Tuning
- 7 Model Evaluation
- 8 Deployment and Monitoring

Reference: Russell & Norvig (2020), Chapter 18

The ML Pipeline Workflow



Key Principle: Iterative process with continuous improvement

Why Build ML Pipelines?

Benefits:

- **Reproducibility:** Consistent results across runs
- **Automation:** Reduces manual effort
- **Scalability:** Handles large datasets
- **Maintainability:** Easier to update and debug
- **Collaboration:** Team can work together
- **Version Control:** Track changes over time

Challenges Without Pipelines:

- Manual, error-prone processes
- Difficult to reproduce results
- Hard to track experiments
- Integration issues
- Inconsistent data handling
- Time-consuming iterations

Important

A well-designed pipeline is crucial for production-ready ML systems.

Reference: Rich et al. (2009). Chapter 17

Data Preprocessing Overview

Definition

Data preprocessing transforms raw data into a clean, structured format suitable for machine learning algorithms.

Why Preprocessing Matters:

- Real-world data is often messy, incomplete, and inconsistent
- ML algorithms require clean, numerical data
- Quality of data directly impacts model performance
- *"Garbage in, garbage out"*

$$\text{Model Performance} = f(\text{Algorithm, Data Quality})$$

Reference: Bishop (2006), Chapter 1

Common Data Quality Issues

1 Missing Values

- Incomplete records, sensor failures, human error
- Can bias results or cause algorithms to fail

2 Noisy Data

- Errors, outliers, inconsistencies
- Measurement errors, data entry mistakes

3 Inconsistent Data

- Different formats, units, naming conventions
- Duplicates, contradictions

4 Imbalanced Data

- Unequal class distributions
- Can lead to biased models

5 Irrelevant Features

- Features that don't contribute to prediction
- Increase complexity without benefit

Reference: Russell & Norvig (2020), Section 18.6

Data Cleaning Techniques

1. Handling Missing Values:

- **Deletion:** Remove rows/columns with missing data
 - Listwise deletion, pairwise deletion
- **Imputation:** Fill in missing values
 - Mean/median/mode imputation
 - Forward fill, backward fill
 - Interpolation methods
 - Model-based imputation (KNN, regression)
- **Indicator Variables:** Create binary flag for missingness

$$x_{\text{imputed}} = \begin{cases} \bar{x} & \text{if } x \text{ is missing (mean imputation)} \\ x & \text{otherwise} \end{cases}$$

Reference: Bishop (2006), Section 9.3

Data Cleaning Techniques (Continued)

2. Handling Noisy Data and Outliers:

- **Statistical Methods:**

- Z-score: Remove points with $|z| > 3$
- IQR method: $Q_1 - 1.5 \times IQR$ to $Q_3 + 1.5 \times IQR$

- **Binning:** Group values into bins/buckets

- **Smoothing:** Moving averages, regression

- **Clustering:** Detect outliers as points far from clusters

3. Handling Duplicates:

- Identify exact duplicates
- Fuzzy matching for near-duplicates
- Remove or merge duplicate records

4. Data Type Conversion:

- Convert strings to appropriate types
- Parse dates and timestamps
- Encode categorical variables

Data Transformation Techniques

1. Scaling and Normalization:

- **Min-Max Scaling:** Scale to range $[0, 1]$

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- **Standardization (Z-score):** Zero mean, unit variance

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

- **Robust Scaling:** Use median and IQR

$$x_{\text{robust}} = \frac{x - \text{median}(x)}{\text{IQR}(x)}$$

When to use:

- Distance-based algorithms (KNN, K-means, SVM)
- Gradient descent optimization
- Neural networks

Reference: Bishop (2006), Section 3.1

Data Transformation Techniques (Continued)

2. Encoding Categorical Variables:

- **Label Encoding:** Assign integer to each category
 - Red $\rightarrow$ 0, Green $\rightarrow$ 1, Blue $\rightarrow$ 2
 - Suitable for ordinal data
- **One-Hot Encoding:** Create binary column for each category
 - Red $\rightarrow$ [1, 0, 0], Green $\rightarrow$ [0, 1, 0], Blue $\rightarrow$ [0, 0, 1]
 - Suitable for nominal data
 - Avoids imposing artificial ordering
- **Target Encoding:** Replace category with target mean
- **Frequency Encoding:** Replace with frequency of occurrence
- **Binary Encoding:** Convert to binary representation

Caution

One-hot encoding can lead to high dimensionality (curse of dimensionality).

Reference: Russell & Norvig (2020), Section 18.6

Definition

Feature engineering is the process of creating new features or transforming existing ones to improve model performance.

Common Techniques:

① Feature Creation:

- Polynomial features: $x_1, x_2 \rightarrow x_1, x_2, x_1^2, x_1x_2, x_2^2$
- Interaction terms: Combine multiple features
- Domain-specific features: Use domain knowledge

② Feature Extraction:

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders for deep feature learning

③ Feature Selection:

- Remove irrelevant or redundant features
- Reduce dimensionality and computational cost

Feature Selection Methods

1. Filter Methods:

- Statistical tests (correlation, chi-square, ANOVA)
- Information gain, mutual information
- Fast, independent of learning algorithm

2. Wrapper Methods:

- Forward selection, backward elimination
- Recursive Feature Elimination (RFE)
- Use model performance as criterion
- Computationally expensive

3. Embedded Methods:

- LASSO (L1 regularization): $\min ||y - X\beta||^2 + \lambda ||\beta||_1$
- Ridge (L2 regularization): $\min ||y - X\beta||^2 + \lambda ||\beta||_2^2$
- Tree-based feature importance
- Built into the learning algorithm

Reference: Bishop (2006), Section 3.1.4

Data Splitting Strategy

Purpose

Split data into separate sets to evaluate model performance on unseen data and prevent overfitting.

Common Splits:

- 1 **Training Set (60-80%):** Learn model parameters
- 2 **Validation Set (10-20%):** Tune hyperparameters, model selection
- 3 **Test Set (10-20%):** Final evaluation, report performance



Important: Never use test data during model development!

Reference: Russell & Norvig (2020), Section 18.4

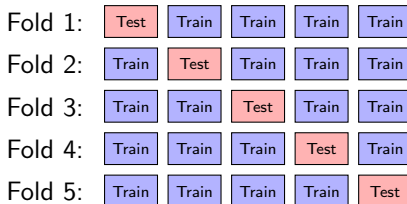
Cross-Validation

Definition

Cross-validation is a technique to assess model performance by training and testing on different subsets of data.

K-Fold Cross-Validation:

- 1 Split data into k equal folds
- 2 Train on $k - 1$ folds, test on remaining fold
- 3 Repeat k times, each fold used once as test set
- 4 Average performance across all folds



Advanced Cross-Validation Techniques

1. Stratified K-Fold:

- Preserve class distribution in each fold
- Important for imbalanced datasets

2. Leave-One-Out (LOO):

- Special case: $k = n$ (number of samples)
- Train on $n - 1$ samples, test on 1
- Computationally expensive but low bias

3. Time Series Cross-Validation:

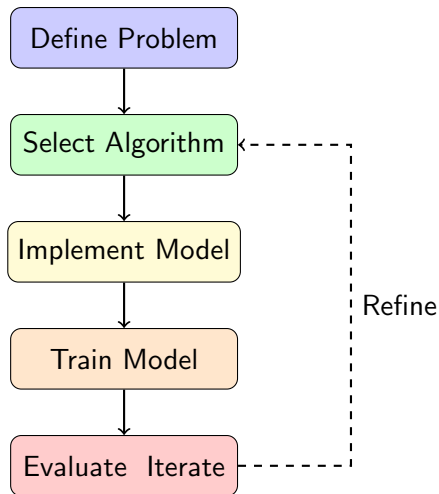
- Forward chaining: respect temporal order
- Train on past, test on future
- No random shuffling

4. Group K-Fold:

- Ensure same group not in both train and test
- Useful for patient data, user data

Reference: Russell & Norvig (2020), Section 18.4

Model Development Process



Reference: Rich et al. (2009), Chapter 17

Problem Definition and Formulation

Key Questions:

① What is the objective?

- Prediction, classification, clustering, recommendation?
- Business goal and success metrics

② What type of learning?

- Supervised, unsupervised, reinforcement learning?
- Classification vs. regression?

③ What are the constraints?

- Computational resources
- Latency requirements
- Interpretability needs

④ How will performance be measured?

- Accuracy, precision, recall, F1-score?
- MSE, MAE, R^2 for regression?
- Business-specific metrics?

Reference: Russell & Norvig (2020), Chapter 18

Algorithm Selection Criteria

Factors to Consider:

① Data Characteristics:

- Size: small vs. large datasets
- Dimensionality: number of features
- Data type: numerical, categorical, text, images
- Linear vs. non-linear relationships

② Problem Type:

- Binary vs. multi-class classification
- Regression vs. classification
- Balanced vs. imbalanced classes

③ Performance Requirements:

- Training time
- Prediction speed
- Memory constraints

④ Interpretability:

- Black-box vs. interpretable models
- Regulatory requirements

Reference: Bishop (2006), Chapter 1

Popular ML Algorithms Overview

Algorithm	Type	Best For
Linear Regression	Regression	Linear relationships
Logistic Regression	Classification	Binary classification, baseline
Decision Trees	Both	Interpretability, non-linear
Random Forest	Both	High accuracy, robust
SVM	Both	High-dimensional, clear margins
K-Nearest Neighbors	Both	Simple, instance-based
Naive Bayes	Classification	Text classification, fast
Neural Networks	Both	Complex patterns, large data
Gradient Boosting	Both	High performance, competitions
K-Means	Clustering	Unsupervised grouping

Reference: Russell & Norvig (2020), Chapters 18-20

Bias-Variance Tradeoff

Key Concept

The bias-variance tradeoff is fundamental to understanding model performance and generalization.

Bias: Error from oversimplified assumptions

- High bias → underfitting
- Model too simple to capture patterns

Variance: Error from sensitivity to training data fluctuations

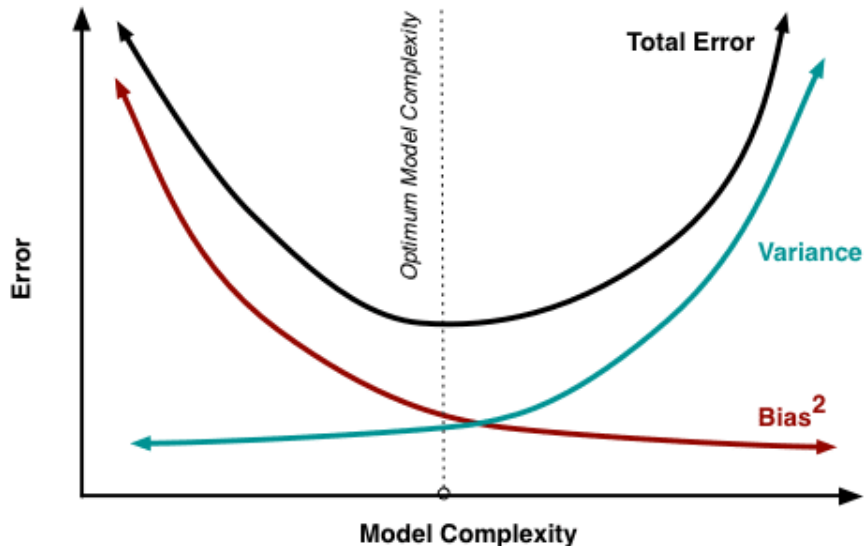
- High variance → overfitting
- Model too complex, memorizes noise

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Goal: Find sweet spot balancing bias and variance

Reference: Bishop (2006), Section 3.2

Bias-Variance Tradeoff Visualization



Handling Underfitting and Overfitting

Underfitting Solutions:

- Increase model complexity
- Add more features
- Reduce regularization
- Train longer
- Use more powerful algorithms
- Engineer better features

Overfitting Solutions:

- Get more training data
- Reduce model complexity
- Use regularization (L1, L2, dropout)
- Feature selection
- Cross-validation
- Early stopping
- Ensemble methods

Key Principle

Always evaluate on held-out test data to detect overfitting.

Reference: Bishop (2006), Section 1.3

Regularization Techniques

Purpose: Prevent overfitting by constraining model complexity

1. L1 Regularization (Lasso):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n |\theta_i|$$

- Produces sparse solutions (feature selection)
- Some coefficients become exactly zero

2. L2 Regularization (Ridge):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n \theta_i^2$$

- Shrinks coefficients toward zero
- Doesn't eliminate features

3. Elastic Net: Combination of L1 and L2

$$J(\theta) = \text{Loss}(\theta) + \lambda_1 \sum_{i=1}^n |\theta_i| + \lambda_2 \sum_{i=1}^n \theta_i^2$$

Reference: Bishop (2006), Section 3.1

Model Training Process

Definition

Training is the process of learning model parameters from training data by optimizing an objective function.

General Training Framework:

$$\theta^* = \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m L(f_{\theta}(x^{(i)}), y^{(i)}) + R(\theta)$$

Where:

- θ : Model parameters
- L : Loss function
- f_{θ} : Model with parameters θ
- $R(\theta)$: Regularization term
- m : Number of training examples

Reference: *Bishop (2006), Chapter 5*

Common Loss Functions

For Regression:

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

- **Huber Loss:** Robust to outliers

Common Loss Functions

For Classification:

- **Cross-Entropy Loss:**

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^k y_j^{(i)} \log(\hat{y}_j^{(i)})$$

- **Hinge Loss:** Used in SVMs

Reference: Bishop (2006), Sections 4.3, 5.2

Gradient Descent: Iteratively update parameters

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Variants:

① Batch Gradient Descent:

- Use all training data per update
- Slow but stable

② Stochastic Gradient Descent (SGD):

- Use one sample per update
- Fast but noisy

③ Mini-Batch Gradient Descent:

- Use small batch per update
- Balance between speed and stability

Reference: Bishop (2006), Section 5.2

Adaptive Learning Rate Methods:

1. Momentum:

$$v_{t+1} = \beta v_t + \eta \nabla_{\theta} J(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}$$

- Accelerates convergence, dampens oscillations

2. Adam (Adaptive Moment Estimation):

- Combines momentum and adaptive learning rates
- Most popular for deep learning
- Automatically adjusts learning rate per parameter

3. RMSprop:

- Adapts learning rate based on gradient magnitudes
- Good for non-stationary objectives

Reference: Russell & Norvig (2020), Section 18.7

Model Evaluation Metrics - Classification

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metrics:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$ (When model predicts positive, how often correct?)
- **Recall (Sensitivity):** $\frac{TP}{TP+FN}$ (Of all actual positives, how many detected?)
- **F1-Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **Specificity:** $\frac{TN}{TN+FP}$

Reference: Russell & Norvig (2020), Section 18.8

Model Evaluation Metrics - Regression

Common Regression Metrics:

1. Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Interpretable, same units as target

2. Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Penalizes large errors more heavily

3. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

4. R^2 (Coefficient of Determination):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Reference: Bishop (2006), Section 1.5

Hyperparameters vs. Parameters

Parameters:

- Learned from data during training
- Internal to the model
- Examples:
 - Weights in neural networks
 - Coefficients in linear regression
 - Split points in decision trees

Hyperparameters:

- Set before training
- Control learning process
- Examples:
 - Learning rate
 - Number of trees in forest
 - Network architecture
 - Regularization strength

Key Difference

Parameters are optimized during training; hyperparameters are tuned externally.

Reference: Bishop (2006), Chapter 1

Common Hyperparameters by Algorithm

Algorithm	Key Hyperparameters
Linear/Logistic Regression	Regularization (λ), penalty type (L1/L2)
Decision Trees	Max depth, min samples split, criterion
Random Forest	# trees, max depth, max features
SVM	Kernel type, C (penalty), gamma
KNN	K (neighbors), distance metric, weights
Neural Networks	# layers, neurons/layer, activation, dropout
Gradient Boosting	# estimators, learning rate, max depth
K-Means	K (clusters), initialization method

Universal Hyperparameters: Learning rate, batch size, number of epochs

Reference: Russell & Norvig (2020), Chapter 18

Hyperparameter Tuning Methods

1. Manual Search:

- Expert intuition and trial-and-error
- Time-consuming, not systematic

2. Grid Search:

- Define grid of hyperparameter values
- Exhaustively try all combinations
- Pros: Systematic, guaranteed to find best in grid
- Cons: Exponentially expensive with dimensions

Reference: Bishop (2006), Section 1.3

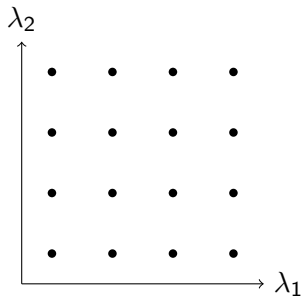
3. Random Search:

- Randomly sample hyperparameter combinations
- Surprisingly effective, often better than grid
- Good for high-dimensional spaces

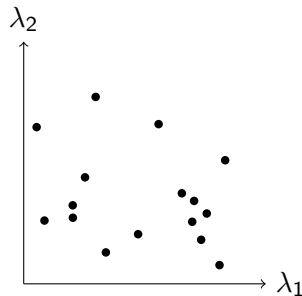
4. Bayesian Optimization:

- Build probabilistic model of objective function
- Intelligently choose next hyperparameters to try
- Most sample-efficient method

Grid Search vs. Random Search



Grid Search (16 points)



Random Search (16 points)

Key Insight: Random search explores more values per hyperparameter, often finding better configurations faster.

Reference: Russell & Norvig (2020), Section 18.4

Hyperparameter Tuning Best Practices

Guidelines:

① Start Simple:

- Begin with default values
- Tune one hyperparameter at a time initially

② Use Validation Set:

- Never tune on test set
- Use cross-validation for robust estimates

③ Search Strategy:

- Coarse to fine: broad search first, then narrow
- Use log scale for learning rate, regularization

④ Computational Budget:

- Balance search thoroughness with time
- Early stopping for poor configurations

⑤ Track Experiments:

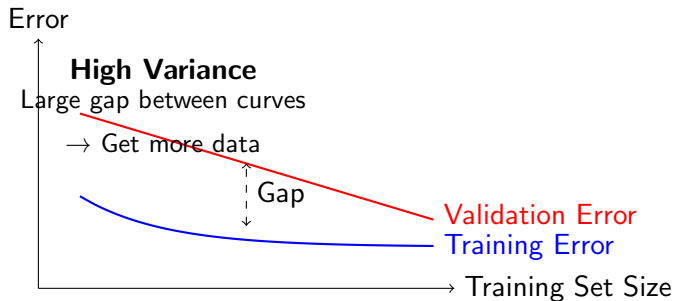
- Log all hyperparameters and results
- Use experiment tracking tools (MLflow, Weights & Biases)

Reference: Rich et al. (2009), Chapter 17

Learning Curves

Purpose

Learning curves show model performance vs. training set size or training iterations, helping diagnose bias and variance.



Reference: Russell & Norvig (2020), Section 18.4

Diagnosing Problems with Learning Curves

High Bias (Underfitting):

- Both curves plateau at high error
- Small gap between them
- More data won't help much
- **Solutions:**
 - Increase model complexity
 - Add features
 - Reduce regularization

High Variance (Overfitting):

- Large gap between curves
- Training error low, validation high
- More data could help
- **Solutions:**
 - Get more data
 - Reduce complexity
 - Increase regularization
 - Feature selection

Ideal Scenario

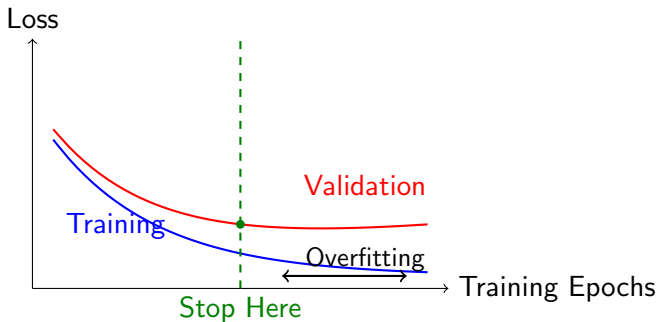
Both curves converge to low error with minimal gap.

Reference: Bishop (2006), Section 1.3

Early Stopping

Concept

Stop training when validation performance stops improving to prevent overfitting.



Implementation: Track validation loss, stop when it increases for n consecutive epochs (patience).

Reference: Bishop (2006) Section 5.2

Principle

Combine multiple models to achieve better performance than individual models.

Main Approaches:

① Bagging (Bootstrap Aggregating):

- Train models on different data subsets
- Average predictions (regression) or vote (classification)
- Example: Random Forest
- Reduces variance

② Boosting:

- Train models sequentially, each correcting previous errors
- Weighted combination of weak learners
- Examples: AdaBoost, Gradient Boosting, XGBoost
- Reduces bias

③ Stacking:

- Train meta-model on predictions of base models
- Learns optimal combination

Complete ML Pipeline Example

Algorithm 1 End-to-End ML Pipeline

- 1: **Data Collection:** Gather and load raw data
 - 2: **Exploratory Analysis:** Understand data distribution and patterns
 - 3: **Data Cleaning:**
 - 4: Handle missing values, outliers, duplicates
 - 5: **Feature Engineering:**
 - 6: Create new features, transform existing ones
 - 7: **Data Splitting:** Train/Validation/Test split
 - 8: **Preprocessing:** Scale features, encode categories
 - 9: **Model Selection:** Choose candidate algorithms
 - 10: **Training:** Fit models on training data
 - 11: **Hyperparameter Tuning:** Optimize on validation set
 - 12: **Evaluation:** Assess performance on test set
 - 13: **Deployment:** Deploy best model to production
 - 14: **Monitoring:** Track performance, retrain as needed
-

Pipeline Automation and Tools

Why Automate?

- Consistency and reproducibility
- Efficiency and scalability
- Easy experimentation
- Production deployment

Popular Tools and Frameworks:

- **Scikit-learn Pipeline:** Chaining transformers and estimators
- **Apache Airflow:** Workflow orchestration
- **Kubeflow:** ML pipelines on Kubernetes
- **MLflow:** Experiment tracking and model registry
- **TensorFlow Extended (TFX):** Production ML pipelines
- **Amazon SageMaker:** End-to-end ML platform
- **Azure ML:** Cloud-based ML pipelines

Reference: Russell & Norvig (2020), Chapter 18

Best Practices for ML Pipelines

Key Recommendations:

① Version Control:

- Track code, data, models, and experiments
- Use Git for code, DVC for data

② Reproducibility:

- Set random seeds
- Document dependencies and environment
- Use containerization (Docker)

③ Modularity:

- Separate preprocessing, training, evaluation
- Reusable components

④ Validation Strategy:

- Never leak test data
- Use cross-validation consistently

⑤ Monitoring:

- Track model performance in production
- Detect data drift and model degradation

Reference: Bishop (2006), Chapter 1

Key Takeaways

1. Preprocessing and Cleaning:

- Critical for model performance
- Handle missing values, outliers, and inconsistencies
- Feature scaling and encoding are essential
- Feature engineering can significantly improve results

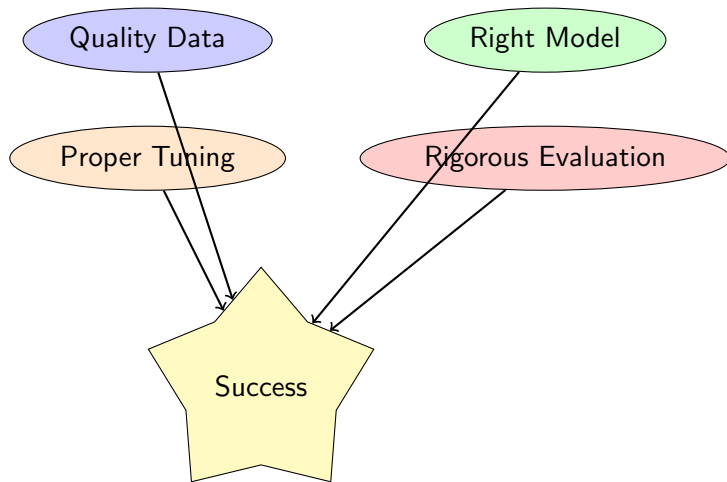
2. Model Development:

- Understand bias-variance tradeoff
- Choose appropriate algorithms for your problem
- Use regularization to prevent overfitting
- Ensemble methods often yield best results

3. Training, Testing, and Tuning:

- Proper data splitting is crucial
- Cross-validation provides robust estimates
- Systematic hyperparameter tuning improves performance
- Monitor learning curves to diagnose problems

ML Pipeline Success Factors



Remember: A machine learning pipeline is only as strong as its weakest component.

Common Pitfalls to Avoid

① Data Leakage:

- Using test data during training or preprocessing
- Including future information in features

② Ignoring Data Quality:

- Skipping exploratory data analysis
- Not handling missing values properly

③ Overfitting:

- Training without validation
- Too complex models for small datasets

④ Poor Evaluation:

- Using wrong metrics
- Not considering business context

⑤ Premature Optimization:

- Tuning hyperparameters before fixing data issues
- Complex models before simple baselines

Reference: Russell & Norvig (2020), Chapter 18

References



Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Chapters 18-20: Machine Learning, Decision Trees, Statistical Learning, Deep Learning



Rich, E., Knight, K., & Nair, S. B. (2009). *Artificial Intelligence* (3rd ed.). McGraw-Hill.

Chapter 17: Learning and Statistical Methods



Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Chapters 1-5, 9, 12: Introduction, Probability, Linear Models, Neural Networks, Clustering, Dimensionality Reduction

Thank You!

Questions?

"In God we trust, all others must bring data."

- W. Edwards Deming